

PRÁCTICA TEMA 0 · INTRODUCCIÓN A PYTHON

Práctica 3: Ventas

Alumno/a: _____ Fecha: _____

Objetivo

Practicar lectura de ficheros CSV, conversión de tipos, funciones, comprensiones, conjuntos, diccionarios, ordenación y contadores a partir de una colección de registros.

Modelo de datos

Trabajaremos con datos de ventas. Cada venta contiene los siguientes campos:

- id: identificador único de la venta (str).
- fecha: fecha de la venta (date).
- cliente: nombre del comprador (str).
- producto: nombre del producto vendido (str).
- cantidad: número de unidades vendidas (int).
- precio: precio unitario en euros (float).
- beneficio: porcentaje de beneficio de la venta (float).

```
from collections import namedtuple
```

```
Venta = namedtuple('Venta', 'id fecha cliente producto cantidad precio beneficio')
```

Un registro podría ser:

```
Venta('2051', date(2019, 6, 4), 'ClienteB', 'ProductoC', 11, 7.60, 2.67)
```

Esta venta corresponde a 11 unidades del ProductoC a 7,60 € cada una, con un beneficio del 2,67 %. Para estos ejercicios se presupone que los datos ya están cargados en una lista de objetos Venta.

Indicaciones

- El fichero ventas.txt forma parte de la práctica y debe utilizarse para cargar los datos antes de resolver los ejercicios.
- Respeta exactamente los nombres de las funciones indicados.
- Cuando el enunciado prohíba bucles, usa una comprensión.
- Puedes crear datos de prueba manualmente para comprobar tus funciones.

Lectura del fichero

El fichero `ventas.txt` se entrega junto con la práctica. Contiene una cabecera y registros separados por comas con el formato `id, fecha, cliente, producto, cantidad, precio, beneficio`.

Implementa la función `leer(nombre)`. Debe abrir el fichero con codificación UTF-8, ignorar la cabecera, convertir cada línea en un objeto `Venta` y devolver una lista con todos los registros. La fecha debe convertirse a `date` con formato `dd/mm/aaaa`; cantidad a `int`; precio y beneficio a `float`.

Ejercicio 1

Implementa `clientes_producto1_y_producto2(ventas, producto1, producto2)`. Recibe una lista de objetos `Venta` y los nombres de dos productos. Debe devolver un conjunto con los nombres de los clientes que hayan comprado ambos productos. No se pueden usar bucles explícitos; si necesitas iterar, utiliza comprensiones.

Ejemplo de interpretación

Si para `'ProductoA'` y `'ProductoC'` se obtiene `{'ClienteA', 'ClienteB'}`, significa que esos clientes aparecen como compradores de los dos productos.

Ejercicio 2

Implementa `diccionario_porcentaje_cantidad_producto(ventas)`. Debe devolver un diccionario que relacione cada producto con el porcentaje de unidades vendidas de ese producto respecto al total de unidades vendidas. Utiliza al menos una función auxiliar.

Ejemplo de interpretación

Si `'ProductoC'` tiene asociado 3.56, sus unidades representan el 3,56 % del total de unidades vendidas.

Ejercicio 3

Implementa `producto_mas_vendido(ventas, clientes, n=3)`. Recibe una lista de ventas, un conjunto de nombres de clientes y un entero `n`. Debe devolver el nombre del producto más frecuente entre las `n` compras con mayor número de unidades realizadas por cualquiera de esos clientes.

Ejemplo de interpretación

Si entre las 5 compras de mayor cantidad de `ClienteA` y `ClienteF` el producto que más se repite es `ProductoC`, la función debe devolver `'ProductoC'`.

Entrega

Entrega un único archivo `.py` con las tres funciones. El código debe ser legible y debe incluir pruebas sencillas de funcionamiento.